

CSCI 5990 Assignment 3

Overview

This project implements a Tetris puzzle solver that places tetromino pieces on a grid to maximize value. The value is calculated as points earned from clearing rows minus the penalty of the remaining grid configuration. The solver supports lookahead functionality, allowing it to consider future pieces when making placement decisions.

Data Structures Used

- 1. 2D Character Array (`char[][] puzzleBoard`):** It uses a 2D character array to represent the game board because it's simple and efficient for checking cell states. Each cell can be either a space character (empty) or a non-space character (filled). This approach allows for $O(1)$ access to any cell, which is crucial when checking collisions and calculating penalties.
- 2. HashMap for Pieces (`HashMap<Integer, Piece>`):** stores pieces in a HashMap with their IDs as keys because it provides $O(1)$ lookup time when placing pieces. This is much more efficient than searching through a list every time we need to find a piece by ID.
- 3. HashMap for Frequencies (`HashMap<Integer, Integer>`):** I kept piece frequencies in a separate HashMap to quickly access the relative frequency of each piece when calculating expected values. This separation of concerns makes the code cleaner and allows for efficient frequency-weighted calculations.
- 4. Pre-computed Rotations (`char[][][] rotations`):** pre-computes all 4 rotations of each piece during initialization rather than calculating them on-the-fly. This trades a small amount of memory for significant time savings, as we check many rotations during the placement search.

Solution Strategy

Core Approach

My solution uses a greedy search with lookahead strategy. For each piece placement, it:

1. Tries all possible rotations (0° , 90° , 180° , 270°)
2. Tries all valid column positions
3. Simulates dropping the piece to find its landing position

4. Calculates the value for each placement
5. Selects the placement with the highest value

Lookahead Implementation

Implemented lookahead using recursive expected value calculation. When lookahead > 0, the algorithm:

1. Calculates the immediate value of placing the current piece
2. Recursively considers all possible next pieces weighted by their frequencies
3. Computes the expected future value: $(\text{freq_A} \times \text{value_A} + \text{freq_B} \times \text{value_B} + \dots) / \text{total_frequency}$
4. Selects the placement that maximizes: $\text{immediate_value} + \text{expected_future_value}$

I chose this approach because it correctly implements the mathematical definition of expected value while being computationally manageable. The recursive structure naturally handles any lookahead depth.

Efficiency Optimizations made to achieve some degree of efficiency:

- 1. Pre-computed Rotations:** Instead of rotating pieces during placement search, I pre-compute all rotations when a piece is added. This eliminates redundant calculations and makes the search $O(1)$ for rotation access.
- 2. Board Copying Strategy:** creates deep copies of the board only when testing placements, not when making the actual placement. This allows me to test many possibilities without modifying the actual game state until I've found the best one.
- 3. Early Termination:** skips invalid placements immediately (e.g., pieces that don't fit in a column) rather than processing them further. This reduces unnecessary computations.
- 4. Efficient Row Clearing:** I used `System.arraycopy()` for shifting rows after clearing, which is much faster than manually copying elements in a loop.

Heuristic Evaluation

used an evaluation system with:

1. **Actual Value:** The true value (points - penalty) that gets returned
2. **Heuristic Value:** An adjusted value used for placement selection

I separated these because the actual value must be mathematically correct for lookahead calculations, while heuristics help guide the search toward better placements. For example, I give large bonuses for row clearing and prefer lower placements that fill gaps, as these strategies generally lead to better long-term outcomes.

Penalty Calculation

calculate penalties efficiently by:

- Iterating through each cell once.
- For filled cells: calculating horizontal distance from center and rows below
- For empty cells: only calculating penalty if they're above the top filled row

This single-pass approach is optimal for the penalty calculation.

Row Clearing Strategy: Identify full rows in one pass, then shift remaining rows down using array copying. The points are calculated based on the number of rows cleared (1=50, 2=100, 3=200, 4=400), chose this approach because it's straightforward and efficient.

Why This Strategy Works

The chosen strategy works well because it balances immediate gains with longer-term planning. When the solver uses a greedy approach with lookahead = 0, it focuses only on the current move. This is effective for immediate optimization because it directly maximizes the value function (points minus penalty), and it remains fast, predictable, and efficient for simple cases. However, when lookahead is greater than 0, the solver becomes more strategic.

The lookahead method evaluates not just the current placement but also considers future possibilities by using expected value calculations that weight outcomes according to piece frequencies. This allows the solver to make decisions that may sacrifice short-term value for better long-term results. The recursive structure naturally supports multiple levels of lookahead, helping the algorithm recognize beneficial future setups.

For example, placing a 2×2 square might result in a slightly worse immediate penalty, but if that placement enables a horizontal bar to clear multiple rows on the following

move, the overall value increases. The lookahead algorithm correctly identifies and prefers such beneficial sequences of moves.

Test Cases

Test Case 1: Basic Placement (Lookahead = 0)

Setup:

- Board: 6×8 grid with some initial filled cells
- Piece: 2×2 square
- Lookahead: 0

Expected: Piece should be placed to minimize immediate penalty, likely near the center and lower on the board.

Returned value: -542, Penalty: 542, Score: 0

Analysis from code: The code correctly places the 2×2 square at the top (columns 0-1) to minimize penalty. The placement is greedy and doesn't consider future pieces, which is correct for lookahead=0. The penalty decreased from 770 to 542, showing the algorithm successfully minimized the immediate penalty.

Test Case 2: Lookahead = 1 with Two Pieces

Setup:

- Board: Initial puzzle state with gaps
- Pieces: 2×2 square (piece 1) and horizontal bar (piece 2), both frequency 1
- Lookahead: 1 for both pieces

Expected: Piece 1 should be placed strategically to set up piece 2 for row clearing. The algorithm should prefer placements that enable future row clearing over immediate penalty minimization.

Value: -1301, Penalty: 752, Score: 100

Analysis from code: The code correctly demonstrates lookahead behavior:

1. Piece 1 is placed at column 3, which increases immediate penalty (from 530 to 672) but sets up the board for piece 2

2. Piece 2 successfully clears 2 rows (score increases from 0 to 100), confirming the lookahead strategy worked
3. The returned value for piece 1 (-1364) includes the expected future value, which is why it's more negative than the immediate penalty would suggest
4. The algorithm correctly prioritized future row clearing over immediate penalty minimization

Test Case 3: Multiple Pieces with Different Frequencies

Setup:

- Board: Initial state
- Pieces: Square (frequency 5), Bar (frequency 15)
- Lookahead: 1

Expected: The expected value calculation should weight the bar's outcomes more heavily ($15/20 = 75\%$) than the square's ($5/20 = 25\%$). When calculating expected future value, placements that work well with the bar should be preferred since it's more likely to appear.

Analysis from code: The code correctly implements frequency-weighted expected value calculation. In the `calculateExpectedValue` method, each piece's best placement value is multiplied by its frequency before summing, then divided by total frequency. This means the bar's outcomes contribute 75% to the expected value, while the square contributes only 25%. The algorithm will prefer placements that maximize value when the bar appears next, which is the correct behavior.

Test Case 4: Row Clearing Points

Setup:

- Board: Nearly full rows
- Pieces: Horizontal bar
- Lookahead: 0

Expected:

- 1 row cleared: 50 points
- 2 rows cleared: 100 points
- 3 rows cleared: 200 points
- 4 rows cleared: 400 points

Analysis from code: The code correctly clears 2 rows and awards 100 points, matching the expected behavior. The points calculation in clearFullRows correctly implements: 1 row = 50, 2 rows = 100, 3 rows = 200, 4 rows = 400. The test confirms the scoring system works as specified.

Test Case 5: Penalty Calculation

Setup:

- Board: Single filled cell at center bottom (row 7, column 2 for 6-column grid)

Expected: Penalty should be relatively low. For a cell at center (column 2 or 3), horizontal distance from center is 0 or 1. At the bottom row, rows below = 0.

Penalty: 1177

Analysis from code: The penalty includes:

- Filled cell penalty: horizontal distance (1) + 10 × rows below (0) = 1
- Empty cell penalties: 7 × rows above for all empty cells above the filled cell
- The total of 1177 accounts for all empty cells above, which is correct according to the penalty formula

Test Case 6: Lookahead = 2

Setup:

- Board: Initial state
- Pieces: Three different pieces
- Lookahead: 2

Expected Behavior: The algorithm should consider two future pieces recursively. This means it will evaluate: current piece -> next piece (considering all possibilities)-> piece after that (considering all possibilities). The returned value should reflect this deeper lookahead.

Analysis from code: When lookahead=2, the code recursively calls calculateExpectedValue with remainingLookahead=1, which then calls it again with remainingLookahead=0.

Test Case 7: Empty Board

Setup:

- Board: Completely empty
- Piece: 2×2 square
- Lookahead: 0

Expected Behavior: Piece should land at the bottom, near the center to minimize penalty.

Penalty: 912

Analysis from code: The code correctly places the piece at the bottom of the board (rows 6-7). The piece is placed at columns 0-1, which is near the left side. The penalty of 912 accounts for:

- Filled cells: horizontal distance from center + rows below
- Empty cells: $7 \times$ rows above for cells above the filled cells
- The placement minimizes penalty by landing at the bottom, which is the correct greedy behavior

Test Case 8: Duplicate Piece Detection

Setup:

- Add a piece
- Try to add the same piece (possibly rotated)

Expected: Should throw `IllegalArgumentException` indicating the piece shape already exists.

Analysis: The code correctly implements duplicate detection in `addPuzzlePiece`. It compares all rotations of the new piece against all rotations of existing pieces. If any rotation matches, it throws an exception. This prevents duplicate pieces from being added, which is important for maintaining correct frequency calculations.

Key Design Decisions

1. Why Separate Actual and Heuristic Values?

I separated these because:

- Actual values must be mathematically correct for expected value calculations
- Heuristics help guide search but shouldn't affect the returned value

- This separation ensures lookahead calculations are accurate while still benefiting from heuristic guidance

2. Why Use HashMap for Pieces?

HashMap provides $O(1)$ lookup by piece ID, which is crucial when placing pieces. A list would require $O(n)$ search, which becomes inefficient with many pieces.

3. Why Recursive Lookahead?

Recursive lookahead naturally handles any depth and correctly implements the expected value formula. An iterative approach would be more complex and harder to verify correctness.

Future Scope:

Looking ahead, there are several enhancements I would have loved to attempt if I had more time, deeper knowledge, and the right resources. One of the biggest improvements I'm interested in is optimizing the solver for deeper lookahead levels, because the computation grows very quickly. With more experience, I would try techniques like pruning, or even parallel processing to handle those heavy calculations efficiently. I also really want to refine the heuristic system; with more understanding of machine learning or genetic algorithms, I would experiment with automatically tuning the heuristic weights instead of setting them manually. Another idea I wish I could explore is adaptive lookahead, where the solver chooses how much to plan ahead based on how complicated the board is.

Conclusion

My solution efficiently solves the Tetris placement problem by combining:

- Pre-computation for speed
- Greedy search for immediate optimization
- Recursive expected value calculation for lookahead
- Efficient data structures for fast access
- Heuristic guidance for better placements